☁️ Cloud Computing i GCP — BigQuery, Dataproc, data lake vs warehouse

Twój laptop ma 16 GB RAM. BigQuery przeskanuje 16 TB w 8 sekund. Jak? 🚀

Python w Zastosowaniach Big Data | Zajęcia nr 11

Prowadzący: Pavlo Melnyk

🔁 Nawiązanie do Wykładu 10

Na ostatnich zajęciach (**Zajęcia nr 10**, 08.12.2026) tunowaliśmy hiperparametry w Spark MLlib — `ParamGridBuilder`, `CrossValidator`, żeby wycisnąć z modelu maksimum jakości.

Ale co, jeśli Twój `RandomForestClassifier` musi trenować na **50 TB danych**, a Twój laptop ma... no właśnie, 16 GB RAM? 😅

> Dziś: przenosimy nasze Big Data (i chęć do analizy) **do chmury** ☁️

📋 Plan wykładu

1. Dlaczego w ogóle chmura dla Big Data?
2. Modele usług — IaaS / PaaS / SaaS
3. Google Cloud Platform — ekosystem Big Data
4. Data lake vs data warehouse vs lakehouse
5. BigQuery — architektura serverless data warehouse
6. Koszty w BigQuery — bytes scanned, partycjonowanie, clustering
7. Łączenie z BigQuery z Pythona i PySpark
8. Dataproc — managed Spark/Hadoop w chmurze
9. Optymalizacja kosztów i typowe pułapki
10. Co dalej — przerwa świąteczna i Streaming

> 🎯 **Po tym wykładzie:** Zaprojektujesz architekturę analityczną w GCP i napiszesz zapytanie SQL świadome kosztów!

💡 Hook — Dlaczego tu jesteśmy?

Wyobraź sobie, że Twoja firma zbiera dane o **20 milionach przejazdów taksówek rocznie**. Chcesz odpowiedzieć na pytanie: *"Jaka jest średnia stawka o każdej godzinie dnia?"*

```python
# Podejście "kup większy laptop" 💸
import pandas as pd
df = pd.read_csv("taxi_trips_20_milionow_wierszy.csv")  # 💥 MemoryError
```

```sql
-- Podejście "chmura" ☁️ — to samo pytanie, inaczej
SELECT EXTRACT(HOUR FROM trip_start_timestamp) AS godzina,
       AVG(fare) AS srednia_stawka
FROM `bigquery-public-data.chicago_taxi_trips.taxi_trips`
GROUP BY godzina
ORDER BY godzina;
-- ⏱️ Wynik w ~8 sekund, zero serwerów do zarządzania
```

> Witaj w świecie **BigQuery**! 📊

🌐 Co to jest Cloud Computing?

Cloud Computing — dostarczanie zasobów IT (serwery, storage, bazy danych, ML) przez internet, na żądanie, w modelu **pay-as-you-go**.

Kryterium	Własne serwery (on-premise)	Chmura
Analogia	Kupno domu 🏡	Wynajem mieszkania 🏢
Koszt początkowy	Ogromny 💸	Zerowy ✅
Skalowanie	Dobuduj piętro? 😅	Kliknij "więcej"
Utrzymanie	Ty naprawiasz dach 🔧	Zarządca budynku
Płatność	Za posiadanie	Za **zużycie**

> 🤔 **Think about it:** Chicago Data Portal publikuje ~20 mln nowych wierszy taksówek rocznie. Kupujesz serwer na to, czy wynajmujesz moc obliczeniową na godzinę zapytania?

📦 Modele usług — IaaS / PaaS / SaaS

Trzy warstwy, jak pizza! 🍕

Model	Co dostajesz?	Twoja odpowiedzialność	Przykłady w GCP
IaaS	Serwery, sieć, storage	OS, runtime, aplikacja	Compute Engine
PaaS	+ OS + runtime	Tylko aplikacja i dane	**Dataproc**
SaaS	Wszystko gotowe	Tylko korzystasz!	**BigQuery**

Analogia pizzowa 🍕: IaaS = kupujesz składniki i pieczesz sam, PaaS = dostajesz ciasto i składniki, SaaS = pizza z dostawą, jesz i tyle 😋

🤔 Think about it — Który to model?

1. 🖥️ Wynajmujesz wirtualną maszynę i instalujesz na niej Sparka ręcznie
2. 📊 Otwierasz BigQuery i piszesz `SELECT ... FROM chicago_taxi_trips`
3. 🔥 Tworzysz klaster Dataproc i submitujesz job PySpark

🚀 Dlaczego chmura dla Big Data?

Problem Big Data	Rozwiązanie chmurowe
20 mln wierszy taksówek + rosną co dzień	Cloud Storage / BigQuery — płać za tyle, ile masz
Analiza raz w miesiącu, ale musi być szybka	Klaster Dataproc na 2h, potem — kasujesz
Nie wiesz z góry, ile mocy obliczeniowej potrzebujesz	Autoskalowanie — od 2 do 200 workerów
Budżet studencki / startupowy	Free tier: 1 TB zapytań BigQuery / miesiąc za darmo

> 💡 **Kluczowe:** W chmurze płacisz za **zużycie**, nie za **posiadanie**. To jak Uber vs własne auto 🚗

🌍 Wielka Trójka (Big Three)

Cecha	GCP 🔵	AWS 🟠	Azure 🔷
Spark managed	**Dataproc**	EMR	HDInsight
Data Warehouse	**BigQuery**	Redshift	Synapse
Object Storage	**GCS**	S3	Blob Storage
Streaming	**Pub/Sub**	Kinesis	Event Hubs
Darmowy tier	$300 / 90 dni	12 mcy free tier	$200 / 30 dni

> 🎯 **Na tym kursie** używamy **GCP** — świetne narzędzia do Big Data, publiczne datasety (jak nasz Chicago Taxi Trips!) i darmowe kredyty dla studentów 🎓

🔵 GCP — Ekosystem Big Data w skrócie

Usługa	Rola	Analogia
Cloud Storage (GCS)	Data lake — surowe pliki	Ogromna szafa na wszystko
BigQuery	Serverless data warehouse — SQL na petabajtach	Bibliotekarz, który zna każdą książkę
Dataproc	Managed Spark/Hadoop	Twój znajomy klaster PySpark, ale w chmurze
Dataflow	Managed Apache Beam (batch + streaming)	Taśmociąg ETL, sam się skaluje

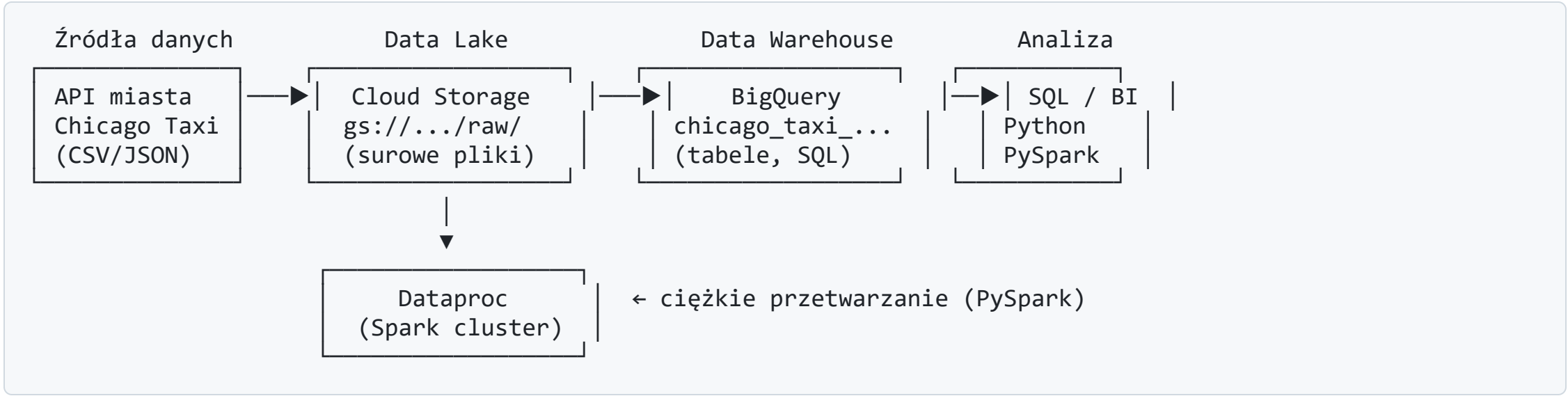
Dziś skupiamy się na **trzech pierwszych** — Dataflow poznamy szerzej przy Streamingu (Wykład 12)!

Data Lake vs Data Warehouse vs Lakehouse

Cecha	Data Lake	Data Warehouse	Lakehouse
Dane	Surowe, dowolny format (JSON, CSV, logi)	Ustrukturyzowane, oczyszczone	Oba naraz
Schema	**Schema-on-read**	**Schema-on-write**	Elastyczne
Kto używa	Data engineerowie, ML	Analitycy biznesowi	Obaj
Przykład GCP	Cloud Storage	BigQuery	BigLake / BigQuery external tables
Analogia	Magazyn surowców	Sklep z gotowymi produktami	Magazyn + witryna w jednym

> W praktyce: dane lądują najpierw w **GCS** (lake), potem trafiają — po czyszczeniu — do **BigQuery** (warehouse).

🏞️ Architektura — od surowych danych do insightów



☁️ Cloud Storage (GCS) — nasz Data Lake

Podstawowe koncepty:

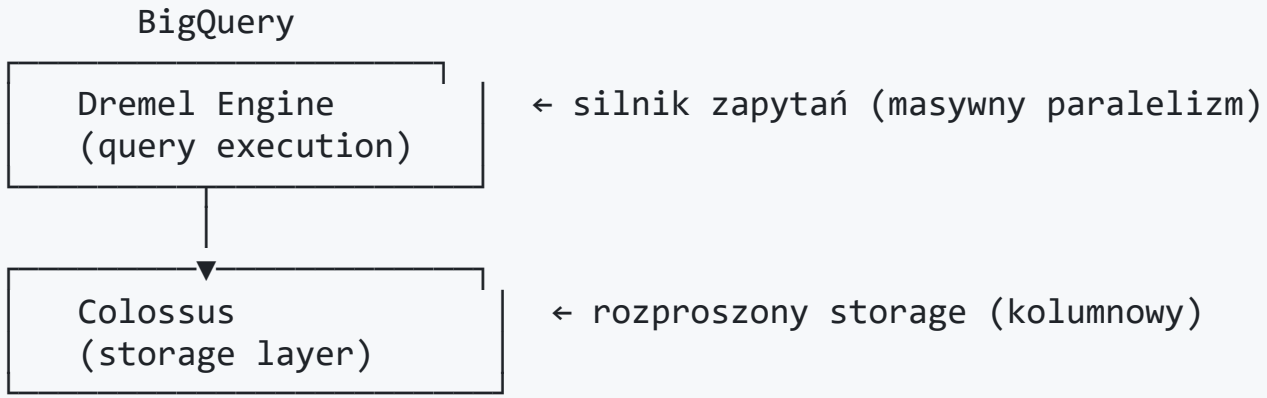
- **Bucket** — kontener na dane, unikalna globalnie nazwa
- **Object** — plik w buckecie (CSV, Parquet, JSON...)
- **URI** — `gs://pjatk-taxi-lake/raw/chicago_taxi_2026.parquet`

Klasa przechowywania	Dostęp	Koszt/GB/msc	Kiedy używać?
Standard	Często	~$0.020	Dane aktywne (bieżący miesiąc przejazdów)
Nearline	1x/msc	~$0.010	Archiwum poprzedniego kwartału
Coldline	1x/kwartał	~$0.004	Dane historyczne do audytu
Archive	1x/rok	~$0.001	Compliance, dane sprzed lat

> 💡 **Pro tip:** Parquet + GCS = idealny duet — kolumnowy format, mniej bajtów do przeczytania!

📊 BigQuery — Serverless Data Warehouse

BigQuery — serverless data warehouse od Google. Odpytujesz **petabajty danych** zwykłym SQL-em, bez zarządzania jakąkolwiek infrastrukturą.



- **Dremel** rozbija zapytanie SQL na tysiące mikro-zadań i wykonuje je równolegle
- **Colossus** przechowuje dane **kolumnowo** — czytasz tylko potrzebne kolumny
- **Bezserwerowy** — nie tworzysz, nie skalujesz, nie łatasz żadnego klastra! 🎉

📊 BigQuery — dlaczego kolumnowo?

```sql
-- Zapytanie potrzebuje TYLKO tych 2 kolumn z 20 dostępnych
SELECT fare, trip_seconds
FROM `bigquery-public-data.chicago_taxi_trips.taxi_trips`
WHERE trip_start_timestamp >= '2023-01-01';
```

Format	Co czyta silnik	Ile bajtów skanuje?
Wierszowy (row-based)	Cały wiersz (wszystkie 20 kolumn)	Dużo — nawet jak potrzebujesz 2 kolumn
Kolumnowy (BigQuery)	Tylko kolumny z `SELECT`	Mało — płacisz tylko za to, co czytasz

> ⚠️ **Dlatego** `SELECT *` w BigQuery to często zbrodnia przeciw portfelowi 💸

📊 BigQuery — publiczny dataset: Chicago Taxi Trips

Google udostępnia dziesiątki **publicznych datasetów** w BigQuery — za darmo dostępny w każdym projekcie GCP:

```sql
-- Ile przejazdów, jaka średnia stawka, per godzina dnia?
SELECT
    EXTRACT(HOUR FROM trip_start_timestamp) AS godzina,
    COUNT(*) AS liczba_przejazdow,
    ROUND(AVG(fare), 2) AS srednia_stawka,
    ROUND(AVG(tips), 2) AS srednia_napiwek
FROM `bigquery-public-data.chicago_taxi_trips.taxi_trips`
WHERE trip_start_timestamp BETWEEN '2023-01-01' AND '2023-01-31'
GROUP BY godzina
ORDER BY godzina;
```

> 🎓 Tabela: `bigquery-public-data.chicago_taxi_trips.taxi_trips` — ponad 200 mln wierszy, dostępna od razu w konsoli BigQuery!

💰 Model rozliczeniowy BigQuery

Model	Cena	Kiedy wybrać?
On-demand	$7.50 / TB skanowanych danych	Nieprzewidywalne, okazjonalne zapytania
Flat-rate / Editions	Od stałej stawki miesięcznej	Stałe, intensywne użycie (firma)
Darmowy tier	1 TB zapytań / msc + 10 GB storage	Idealne na start i naukę! 🎓

```
-- 💥 To zapytanie skanuje CAŁĄ tabelę (~150 GB) = ~$0.75, mimo że liczysz 1 liczbę!
SELECT * FROM `bigquery-public-data.chicago_taxi_trips.taxi_trips`;

-- ✅ To samo pytanie, świadome kosztów (2 kolumny zamiast 20+)
SELECT COUNT(*) FROM `bigquery-public-data.chicago_taxi_trips.taxi_trips`;
```

> ⚠️ **Uwaga:** BigQuery liczy za **przeskanowane bajty**, nie za zwrócone wiersze!

💰 Cost awareness — jak sprawdzić koszt PRZED uruchomieniem?

Konsola BigQuery i Python SDK pokazują **dry run** — ile bajtów zeskanuje zapytanie, zanim zapłacisz:

```
from google.cloud import bigquery

client = bigquery.Client()

job_config = bigquery.QueryJobConfig(dry_run=True, use_query_cache=False)
query = """
    SELECT EXTRACT(HOUR FROM trip_start_timestamp) AS godzina, AVG(fare)
    FROM `bigquery-public-data.chicago_taxi_trips.taxi_trips`
    GROUP BY godzina
"""
job = client.query(query, job_config=job_config)

gb_skanowane = job.total_bytes_processed / 1e9
print(f"To zapytanie zeskanuje {gb_skanowane:.2f} GB")  # 👈 zero kosztów za dry run!
print(f"Szacowany koszt: ${gb_skanowane / 1000 * 7.5:.4f}")
```

🗂️ Partycjonowanie tabel (partitioning)

Partycjonowanie — dzielenie tabeli na fizyczne segmenty (np. po dacie), tak by zapytanie czytało tylko potrzebny fragment.

```sql
-- Tworzenie tabeli partycjonowanej po dacie przejazdu
CREATE TABLE moj_projekt.taxi.trips_partitioned
PARTITION BY DATE(trip_start_timestamp)
AS SELECT * FROM `bigquery-public-data.chicago_taxi_trips.taxi_trips`;

-- Zapytanie czyta TYLKO partycję ze stycznia 2023 — nie całą tabelę!
SELECT AVG(fare)
FROM moj_projekt.taxi.trips_partitioned
WHERE DATE(trip_start_timestamp) BETWEEN '2023-01-01' AND '2023-01-31';
```

> 💡 Partycja = jak rozdziały w książce. Szukasz rozdziału 5? Nie czytasz całej książki od nowa!

🗂️ Clustering tabel

Clustering — sortowanie danych wewnątrz partycji według wybranych kolumn (np. dzielnica odbioru pasażera), by przyspieszyć filtrowanie.

```
CREATE TABLE moj_projekt.taxi.trips_clustered
PARTITION BY DATE(trip_start_timestamp)
CLUSTER BY pickup_community_area
AS SELECT * FROM `bigquery-public-data.chicago_taxi_trips.taxi_trips`;

-- Filtr po pickup_community_area jest teraz dużo tańszy!
SELECT COUNT(*)
FROM moj_projekt.taxi.trips_clustered
WHERE pickup_community_area = 8
  AND DATE(trip_start_timestamp) = '2023-06-15';
```

> 🔑 **Partycja** = duże bloki (data), **clustering** = drobne sortowanie wewnątrz bloku (kategoria).

🤔 Think about it — Ile to kosztuje?

Tabela `taxi_trips` waży **150 GB**. Masz do wyboru 3 zapytania:

1. `SELECT * FROM taxi_trips` (bez partycji)
2. `SELECT fare, tips FROM taxi_trips` (2 kolumny z 23, bez partycji)
3. `SELECT fare FROM trips_partitioned WHERE DATE(...) = '2023-06-15'` (1 dzień z partycji)

Uszereguj od najdroższego do najtańszego! 🤔

BigQuery z Pythona — `google-cloud-bigquery`

```
# !pip install -q google-cloud-bigquery pandas-gbq

from google.cloud import bigquery

client = bigquery.Client(project="moj-projekt-pjatk")

query = """
    SELECT EXTRACT(HOUR FROM trip_start_timestamp) AS godzina,
           ROUND(AVG(fare), 2) AS srednia_stawka
    FROM `bigquery-public-data.chicago_taxi_trips.taxi_trips`
    WHERE trip_start_timestamp BETWEEN '2023-01-01' AND '2023-01-07'
    GROUP BY godzina
    ORDER BY godzina
"""

df = client.query(query).to_dataframe()  # 👈 wynik od razu jako pandas DataFrame!
print(df.head())
```

> 🔑 `to_dataframe()` — most między BigQuery a całym ekosystemem pandas / matplotlib / scikit-learn!

BigQuery w Google Colab — `%%bigquery` magic

W Colabie nie musisz nawet importować klienta ręcznie:

```
from google.colab import auth
auth.authenticate_user()  # 👈 logowanie do Twojego konta Google

%load_ext google.cloud.bigquery
```

```
%%bigquery wyniki --project moj-projekt-pjatk
SELECT payment_type, COUNT(*) AS liczba, ROUND(AVG(fare), 2) AS srednia_stawka
FROM `bigquery-public-data.chicago_taxi_trips.taxi_trips`
WHERE trip_start_timestamp BETWEEN '2023-01-01' AND '2023-03-31'
GROUP BY payment_type
ORDER BY liczba DESC
```

```
wyniki.plot(x="payment_type", y="srednia_stawka", kind="bar")  # 👈 df gotowy!
```

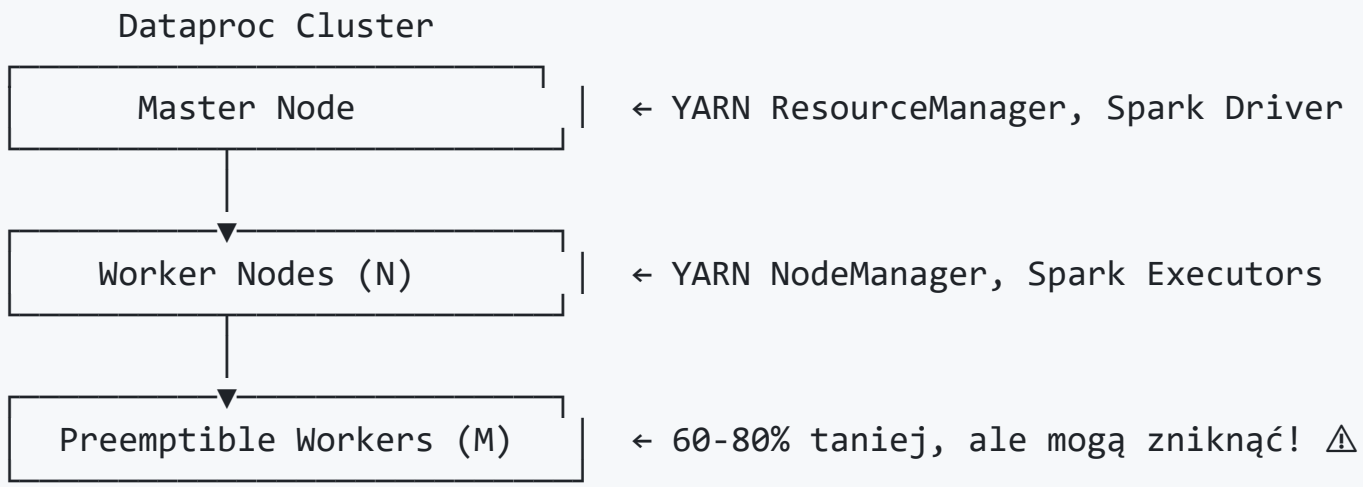
🔥 Dataproc — Managed Spark/Hadoop w GCP

Dataproc — w pełni zarządzany serwis do uruchamiania **Apache Spark**, Hadoop, Flink i Presto.

Kryterium	Spark na własnych VM (DIY)	Dataproc (managed)
Czas setupu	Godziny/dni 😩	**~90 sekund** ⚡
Skalowanie	Ręczne dodawanie nodów	**Autoscaling** 🔄
Koszt	Płacisz nawet, jak stoi	**Ephemeral clusters** — kasuj po pracy!
Monitoring	Konfiguruj sam	Wbudowane dashboardy

> 🤔 **Think about it:** Analiza `chicago_taxi_trips` przez PySpark trwa 40 minut dziennie. Po co utrzymywać klaster 24/7?

🔥 Dataproc — Architektura klastra



- **Master** — zarządza klastrem
- **Workers** — wykonują obliczenia
- **Preemptible Workers** — tańsze, ale Google może je odebrać w każdej chwili

🔥 Dataproc — Tworzenie klastra (gcloud CLI)

```
gcloud dataproc clusters create pjatk-taxi-cluster \
    --region=europe-central2 \
    --master-machine-type=n1-standard-4 \
    --num-workers=2 \
    --worker-machine-type=n1-standard-4 \
    --num-secondary-workers=2 \
    --secondary-worker-type=preemptible \
    --image-version=2.1-debian11 \
    --enable-component-gateway \
    --optional-components=JUPYTER \
    --max-idle=1h
```

Kluczowe parametry:

- `--num-secondary-workers` — preemptible (tańsze!) 💰
- `--enable-component-gateway` — dostęp do Spark UI z przeglądarki 🌐
- `--max-idle=1h` — klaster sam się usuwa po godzinie bezczynności 🧹

🔥 Dataproc — Job PySpark na `taxi_trips`

Plik `analiza_taxi.py` uruchamiany na klastrze — czyta dane wyeksportowane z BigQuery do GCS:

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("AnalizaTaxi").getOrCreate()

df = spark.read.parquet("gs://pjatk-taxi-lake/raw/chicago_taxi_2023.parquet")

wyniki = (df.groupBy("pickup_community_area")
            .agg(F.count("*").alias("liczba_przejazdow"),
                 F.avg("fare").alias("srednia_stawka"))
            .orderBy(F.desc("liczba_przejazdow")))

wyniki.write.parquet("gs://pjatk-taxi-lake/results/", mode="overwrite")
spark.stop()
```

```
gcloud dataproc jobs submit pyspark gs://pjatk-taxi-lake/jobs/analiza_taxi.py \
    --cluster=pjatk-taxi-cluster --region=europe-central2
```

🔗 BigQuery + PySpark na Dataproc

Integracja przez `spark-bigquery-connector` — zero eksportu do plików pośrednich:

```
# Czytanie danych bezpośrednio z BigQuery do Spark DataFrame
df = (spark.read
      .format("bigquery")
      .option("table", "bigquery-public-data.chicago_taxi_trips.taxi_trips")
      .load())

wyniki = df.groupBy("payment_type").count()

# Zapis wyników z powrotem do BigQuery
(wyniki.write
    .format("bigquery")
    .option("table", "moj_projekt.taxi.wyniki_platnosci")
    .option("temporaryGcsBucket", "pjatk-taxi-lake-temp")
    .mode("overwrite")
    .save())
```

> 🔑 Connector wymaga bucketa GCS jako storage tymczasowy dla danych pośrednich.

💰 Optymalizacja kosztów — Dataproc

1. Preemptible/Spot Workers 🎯

60-80% taniej — idealne do batch processing (jak nasza analiza nocna taksówek)

2. Ephemeral Clusters ⏱️

```
gcloud dataproc clusters create temp-cluster ...
gcloud dataproc jobs submit pyspark ...
gcloud dataproc clusters delete temp-cluster --quiet  # 👈 natychmiast!
```

3. Scheduled Deletion 🗑️

```
gcloud dataproc clusters create pjatk-cluster --max-idle=2h
```

4. Odpowiedni rozmiar maszyn

Nie bierz `n1-highmem-96`, jeśli wystarczy `n1-standard-4` 😄

🤔 Think about it — Klaster stały czy ephemeral?

Job PySpark analizujący `taxi_trips` trwa **40 minut**, raz dziennie.

Opcja A: Klaster stały (24/7, 3 workery) → ~$2 160/msc
Opcja B: Ephemeral cluster (40 min/dzień, 6 workerów) → ~$95/msc

Obie dają ten sam wynik. Którą wybierasz? 🤔

> Odpowiedź: **Opcja B** — ponad 20x tańsza, a przy okazji szybsza (2x więcej workerów)! 🏎️💨

⚠️ Najczęstsze pułapki w GCP

Pułapka	Dlaczego to problem	Rozwiązanie
`SELECT *` na dużych tabelach	Skanuje wszystkie kolumny = $$$	Wybieraj konkretne kolumny
Zapomniany klaster Dataproc	Płacisz 24/7 za bezczynność	`--max-idle`, ephemeral clusters
Brak partycjonowania	Każde zapytanie czyta całą tabelę	`PARTITION BY`, `CLUSTER BY`
Publiczny bucket GCS	Wyciek danych, ryzyko bezpieczeństwa	IAM, zasady dostępu, private by default
Brak dry run przed dużym zapytaniem	Niespodziewany rachunek	`dry_run=True` w `QueryJobConfig`

Bezpieczeństwo — szybki przegląd

- **IAM (Identity and Access Management)** — kto ma dostęp do czego (zasada najmniejszych uprawnień!)
- **Service Account** — "konto techniczne" dla aplikacji/skryptów, nie dla ludzi
- **Buckety GCS** — domyślnie prywatne, świadomie nadawaj publiczny dostęp
- **BigQuery row-level security** — ograniczanie widoczności wierszy per użytkownik

> Nigdy nie commituj kluczy `service-account.json` do repozytorium Git! (sprawdź `.gitignore`)

🛠️ Demo — Pełny workflow: od CSV do wyniku

```
# 1. Bucket na dane
gsutil mb -l europe-central2 gs://pjatk-taxi-lake/

# 2. Upload danych
gsutil cp chicago_taxi_sample.parquet gs://pjatk-taxi-lake/raw/

# 3. Klaster Dataproc (ephemeral, max-idle)
gcloud dataproc clusters create demo-cluster \
    --region=europe-central2 --num-workers=2 --max-idle=1h

# 4. Submit joba PySpark
gcloud dataproc jobs submit pyspark \
    gs://pjatk-taxi-lake/jobs/analiza_taxi.py \
    --cluster=demo-cluster --region=europe-central2

# 5. Sprzątanie! 🧹
gcloud dataproc clusters delete demo-cluster --region=europe-central2 --quiet
```

> ⚠️ **ZAWSZE usuwaj klaster po pracy!** Zapomniany klaster to $$$$ 💸

Kluczowe wnioski

1. ✅ Chmura = płacisz za **zużycie**, nie za **posiadanie** sprzętu
2. ✅ **BigQuery** — serverless, kolumnowy, płacisz za **przeskanowane bajty**
3. ✅ **Partycjonowanie i clustering** drastycznie obniżają koszty zapytań
4. ✅ **Dataproc** = Spark, do którego jesteś przyzwyczajony, ale bez zarządzania infrastrukturą
5. ✅ Data lake (GCS) + data warehouse (BigQuery) = solidna architektura Big Data w chmurze

🔮 Co dalej?

Na dzisiejszych **ćwiczeniach** przeanalizujesz przejazdy taksówek w Chicago — realny publiczny dataset BigQuery, z pełnym fallbackiem offline (bez potrzeby konta GCP)! 🚕

Zajęcia nr 12 (12.01.2027): Przetwarzanie strumieniowe — **Structured Streaming i Kafka**

> ❄️ **Przerwa świąteczna!** Nie ma zajęć **22.12.2026** ani **29.12.2026**. Do zobaczenia dopiero w Nowym Roku — Wesołych Świąt i szczęśliwego 2027! 🎄🎉

❓ Pytania?

Dziękuję za uwagę! Do zobaczenia na ćwiczeniach 🚕📊

Prowadzący: Pavlo Melnyk (melnyk@pjwstk.edu.pl)